



Universidad
Indoamérica

UNIT 3 · FUNDAMENTALS OF OPERATING SYSTEMS

Files and file systems

Universidad Tecnológica Indoamérica Lab TD2

How are files accessed? Two levels

On disk (persistent)

- File system structure: superblock, inode tables, blocks
- Inodes: the metadata of every file
- Directories: *name* → *inode* lists

In the kernel (while it is open)

- Descriptor table, one per process
- Open file table, one for the whole system
- In-memory inode table (and the page cache)

File system structure



NVMe disk →

ext4: the block group repeats along the partition (with superblock copies)

Real output

```
$ df -iT . | cut -c1-80; stat -f -c 'block size: %S bytes' .  
Filesystem      Type  Inodes   IUsed   IFree IUse% Mounted on  
/dev/nvme0n1p5 ext4 25600000 2372366 23227634   10% /  
block size: 4096 bytes
```

A file = content + inode

- The **inode** stores: type and permissions, hard links, size, device, UID/GID, timestamps and where the blocks are
- It does **not** store the name: the name is in the directory
- **Fix to the original:** `ctime` is not creation but the last *inode change*. Creation (*Birth*) exists in ext4 and is read with `statx()`.

Real output

```
$ stat notas.txt
  File: notas.txt
  size: 11          Blocks: 8          IO Block: 4096
regular file
Device: 259,5   Inode: 12190686   Links: 2
Access: (0664/-rw-rw-r--)  Uid: ( 1000/student)   Gid: (
1000/student)
Access: 2026-09-30 13:12:49.872600645 -0500
Modify: 2026-09-30 13:12:49.872600645 -0500
Change: 2026-09-30 13:12:49.879601063 -0500
 Birth: 2026-09-30 13:12:49.872600645 -0500
```

From the inode to the blocks: pointers and extents

inode → extent tree (depth 1, 340 entries per block)

	logical	length	physical
#0	0	32,768	34,816
#1	32,768	32,768	67,681
#2	65,536	32,768	100,546
#3	98,304	32,768	133,411
#4	131,072	32,768	166,276
#5	163,840	9,111	199,141

physical block =
 $199,141 + (172,950 - 163,840)$
 = 208,251

index blocks read: 1

the same file with pointers (ext3):

172,951 pointers in 170 index blocks · this offset: 2 index reads · with extents: 6 × 12 bytes

676 MiB



the file in logical blocks (0 ... 172,951); each band is an extent

A name points to an inode

Real output

```
$ ln notas.txt copia.txt; ln -s notas.txt atajo; ls -li notas.txt copia.txt atajo
12190687 lrwxrwxrwx 1 student student 9 Sep 30 13:12 atajo → notas.txt
12190686 -rw-rw-r-- 2 student student 11 Sep 30 13:12 copia.txt
12190686 -rw-rw-r-- 2 student student 11 Sep 30 13:12 notas.txt
```

Hard link (`link`)

Another entry to the **same** inode: same number, `nlink = 2`. Not to directories nor across file systems.

Symbolic link (`symlink`)

A **new** inode whose content is a path (9 bytes: `notas.txt`). It can dangle.

Resolving paths, linking and deleting

\$ cat atajo

relative path: starts at the current directory, inode 1402

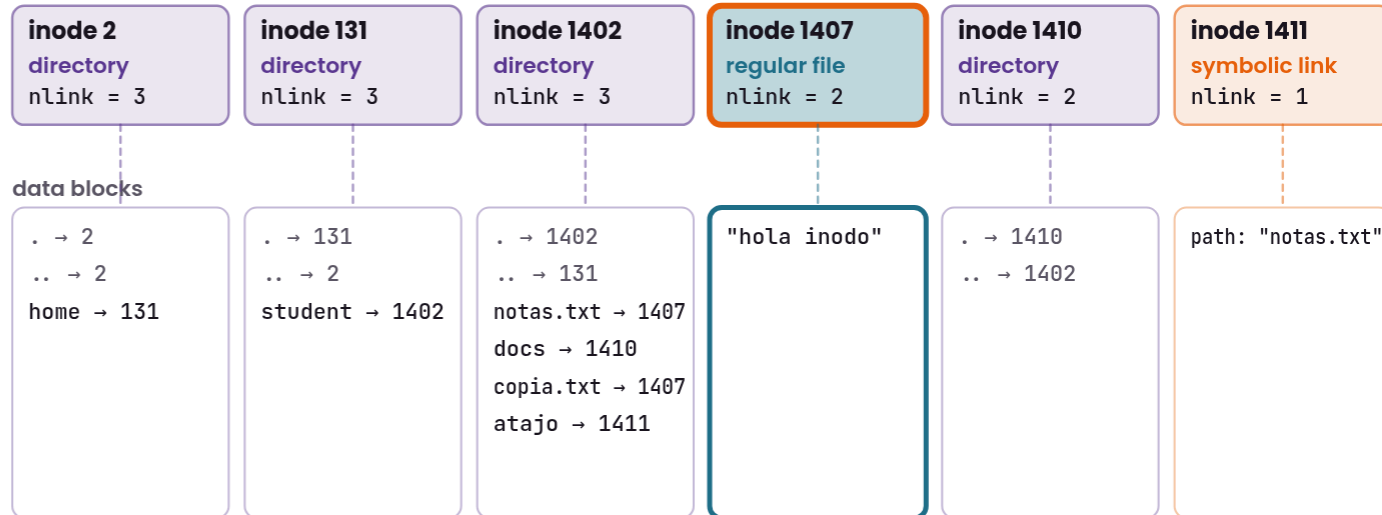
read dir 1402: "atajo" → inode 1411

symbolic link #1: continue with "notas.txt"

read dir 1402: "notas.txt" → inode 1407

read the data blocks of inode 1407: "hola inodo"

inode table



"hola inodo"

Reading the inode: stat, fstat, lstat

C

```
int stat(const char *path, struct stat *st);
int fstat(int fd, struct stat *st);
int lstat(const char *path, struct stat *st); /* does not follow links */

struct stat {
    dev_t    st_dev;    ino_t    st_ino;
    mode_t    st_mode;    nlink_t st_nlink;
    uid_t    st_uid;    gid_t    st_gid;
    dev_t    st_rdev;    /* device, if special file */
    off_t    st_size;    blkcnt_t st_blocks;
    blksize_t st_blksize; /* preferred I/O block size */
    struct timespec st_atim, st_mtim, st_ctim;
};
```

- 0 on success; -1 and `errno` otherwise:
- `EACCES` no search permission (`x`) on a directory of the path
- `ENOENT` does not exist · `ENOTDIR` a component is not a directory
- `EL00P` more than 40 symbolic links
- 2026: times with nanoseconds (`st_mtime = st_mtim.tv_sec`); `statx()` adds creation time.

The type is in `st_mode`: `S_IS...` macros

<code>S_ISREG</code>	regular file	-
<code>S_ISDIR</code>	directory	d
<code>S_ISLNK</code>	symbolic link	l
<code>S_ISCHR</code> · <code>S_ISBLK</code>	character · block device	c · b
<code>S_ISFIFO</code> · <code>S_ISSOCK</code>	pipe · socket	p · s

Real output

```
$ ./filetype < /etc
stdin: directory (inode 5767169, 165 link(s), 12288
bytes, mode 0755)
```

Real output

```
$ echo hi | ./filetype
stdin: FIFO (pipe) (inode 859154, 1 link(s), 0 bytes,
mode 0600)
```

Real output

```
$ ./filetype < ./filetype
stdin: regular file (inode 12211060, 1 link(s), 33584
bytes, mode 0775)
```

Permissions: `access` ... and its trap

C

```
int access(const char *path, int mode); /* R_OK W_OK X_OK F_OK */
```

- -1 if it does not exist or permission is denied: `EROFS` (read-only), `ETXTBSY` (executable in use)...
- It checks with the *real* UID, not the effective one

TOCTOU race (2026 update)

Between `access()` and `open()` another process can swap the file for a link to `/etc/passwd`. The safe way is to **open and handle the error** (`EACCES`), or `openat2()` with `RESOLVE_NO_SYMLINKS`. TD8 (`agentd`) uses `realpath` and Landlock for this reason.

Creating, linking and deleting names

C

```
int mknod(const char *path, mode_t mode, dev_t dev);
int mkfifo(const char *path, mode_t mode); /* S_IFIFO */
int link(const char *old, const char *new);
int symlink(const char *target, const char *path);
int unlink(const char *path); /* EISDIR */
int rmdir(const char *path); /* must be emp
```

- `mknod` creates any type: the type is ORed with the permissions
(`S_IFREG|S_IRUSR|S_IXUSR` = `S_IFREG|0500`);
today it is used almost only for devices. For files: `open(O_CREAT)` ; for pipes: `mkfifo` .
- The requested permissions go through the process `umask` (typically `022` or `002`).
- `unlink` removes an *entry*, not the file.

Opening or creating: `open`

C

```
int open(const char *path, int flags, ... /* mode_t mode */);  
int openat(int dirfd, const char *path, int flags, ...);
```

- Access: `O_RDONLY`, `O_WRONLY`, `O_RDWR`
- Creation: `O_CREAT`, `O_EXCL` (fails if it exists), `O_TRUNC`
- Status: `O_APPEND` (every `write` at the end, atomically)
- Returns the **lowest free descriptor** (≥ 0) or `-1`

Today also

- `O_CLOEXEC` : do not inherit the descriptor on `exec` (glibc always uses it, see it in `strace`)
- `O_NOFOLLOW`, `O_DIRECTORY`, `O_TMPFILE`
- glibc implements `open` with `openat(AT_FDCWD, ...)`; `creat()` is `open(..., O_WRONLY|O_CREAT|O_TRUNC)`

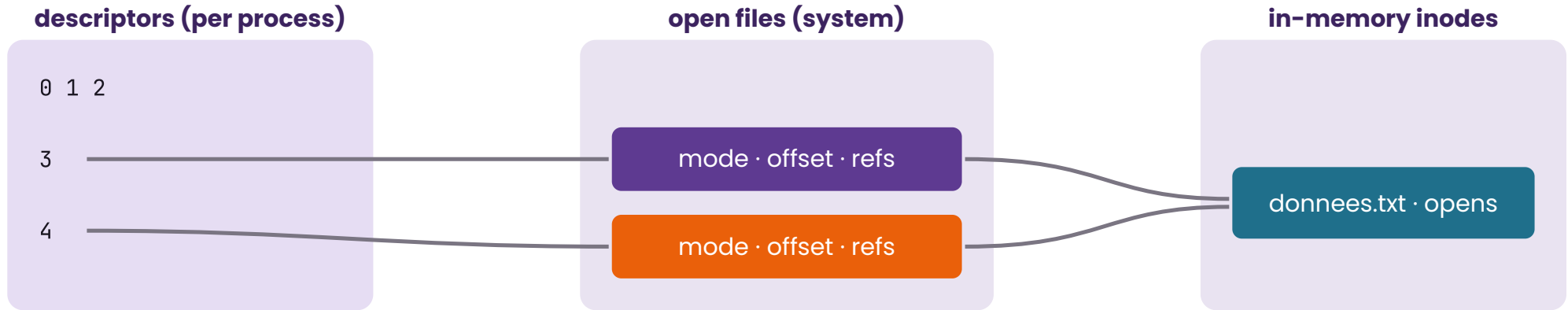
close : when does a file disappear?

When `nlink = 0` **and** no descriptor has it open. Until then, deleted but alive:

Real output

```
$ exec 3< notas.txt; rm notas.txt copia.txt; readlink /proc/$$/fd/3; cat <&3; ls -l atajo; cat atajo
~/fso/tds/tests/tmp/td2/notas.txt (deleted)
hola inodo
lrwxrwxrwx 1 student student 9 Sep 30 13:12 atajo → notas.txt
cat: atajo: No such file or directory
[exit status 1]
```

File management in the kernel: three tables



The **offset** lives in the middle table: that is why it matters who shares each entry.

Aliasing: two open calls, dup and fork

```
fd = open("donnees.txt", O_RDONLY);
read(fd, buf, 4); write(1, buf, 4);
pid = fork();
/* child */ read(fd, buf, 4); write(1, buf, 4);
/* parent */ wait(NULL); n = read(fd, buf, 4);
read(3) uses entry A: reads "efgh" and its offset becomes 8
```

standard output:
"abcdefgh"

descriptors (per process)

parent

0	→ terminal
1	→ terminal
2	→ terminal
3	
4	

child

0	→ terminal
1	→ terminal
2	→ terminal
3	
4	

open files (system)

entry A

O_RDONLY
offset = 8

refs = 2

in-memory inodes (system)

donnees.txt

open by 1 entry

a	b	c	d	e	f	g	h
---	---	---	---	---	---	---	---

Basic functions, `read` and `write` : review

- Inodes: `stat`, `chmod`, `chown`, `link`, `unlink` ...
- Descriptors: `open`, `read`, `write`, `lseek`, `close`, `fcntl` ...
- `fopen` / `fread` : library, not system calls
(`fileno()`)

`read` may return fewer bytes, 0 at the end, -1/ `EINTR`; `write` may be partial. The example on slide 51 had a bug: A2.4.

Walking directories: `opendir`, `readdir`, `closedir`

C

```
DIR *opendir(const char *name);
struct dirent *readdir(DIR *dir); /* NULL at the end */
int closedir(DIR *dir);

struct dirent {
    ino_t      d_ino;      /* inode number */
    unsigned char d_type;  /* DT_REG, DT_DIR, DT_LNK... */
    char       d_name[256];
    /* d_off, d_reclen: internal */
};
```

- `readdir` returns the entries in the file system's internal order (on ext4, a hash index order), not alphabetical
- It includes `.` and `..`
- **Update:** the original said "do not use the first four fields". Today `d_ino` and `d_type` are useful (they save a `stat`), but `d_type` may be `DT_UNKNOWN`: then `lstat` is needed.

Walking a directory: `myls -lR`

DIR *d = opendir(dir); and then:

C tds/td2/myls.c

```
struct dirent *e;
while ((e = readdir(d)) != NULL) {
    if (n == cap) {
        cap *= 2;
        names = realloc(names, cap * sizeof *names);
        exit_if(names == NULL, "realloc");
    }
    names[n] = strdup(e->d_name);
    exit_if(names[n] == NULL, "strdup");
    n++;
}
exit_if(closedir(d) == -1, "closedir");
```

Real output

```
$ ./myls -lR demo
demo:
drwxrwxr-x   3 student student   4096 2026-09-30 13:12
.
drwxrwxr-x   3 student student   4096 2026-09-30 13:12
..
-rw-rw-r--   1 student student     3 2026-09-30 13:12
a.txt
lrwxrwxrwx   1 student student     5 2026-09-30 13:12
link -> a.txt
prw-rw-r--   1 student student     0 2026-09-30 13:12
pipe
drwxrwxr-x   2 student student   4096 2026-09-30 13:12
sub

demo/sub:
drwxrwxr-x   2 student student   4096 2026-09-30 13:12
.
drwxrwxr-x   3 student student   4096 2026-09-30 13:12
..
```

A special file: the chat FIFO

server

`read() = 0` → ends

client 1

closed

client 2

closed

- the server returns from `open()`
- C1 opens the FIFO for writing
- C2 opens the FIFO for writing
- C1 closes
- C2 closes
- `read() = 0`: the last writer closed → end of file

chat.fifo · kernel buffer: 16 pages × 4 KiB = 64 KiB



↓ `read(fd, buf, 4096)` → server output

hola a todos
 buenas
 ¿alguien en el lab?
 aquí el cliente 2
 chao
 me voy

`read() = 0` → EOF

What the original did not get to tell

Journaling

ext4 and XFS write to a journal first: after a power cut there is no need to check the whole disk.

Copy-on-write

Btrfs and ZFS never overwrite: cheap snapshots and checksums of every block.

Cache and `fsync`

`write` ends in the page cache; only `fsync` guarantees it reached the disk.

"Everything is a file": `/proc`, `/sys`, `/dev` and FUSE use the same `open / read / write` interface.

Solved TD2 + AI challenge

- `filetype` with the terminal, a directory, a file and a pipe
- The FIFO chat, with two terminals and then three
- `alias_open` versus `alias_dup`; `myls -lR`

Bash

```
make -C tds && tds/tests/run.sh td2
```



Universidad
Indoamérica

TAKE-AWAYS

Three questions

1. Where is a file's name, and where is its size?
2. Why does `dup` print "abcdefgh" and two `open` calls "abcdabcd"?
3. When does the kernel free the blocks of a deleted file?